

Lazy Evaluation

Descriere

Mihai este un student la Facultatea de Automatică și Calculatoare. Întrucât este foarte pasionat de Programarea Calculatoarelor și Limbaje de Programare, el decide să înceapă un proiect personal, în care să studieze limbajele de programare, cu accent pe sintaxa acestora.

Pentru început, el realizează că are nevoie să dezvolte o structură de date specială în nodurile căreia să țină minte șiruri de caractere. Aceste șiruri de caractere au unele proprietăți speciale:

- sunt formate numai din caracterele 0 și 1.
- șirurile de caractere trebuie să **nu** conțină substringul **01**.

Mai mult, el își dă seama de faptul că nu trebuie să evalueze toate șirurile existente cu aceste proprietăți, pentru că ele sunt în număr infinit. Astfel, el decide să scrie un program care să primească din când în când un număr de șiruri pe care el dorește să le obțină (șirul de numere terminându-se cu un număr negativ) și printează rezultatele. Pentru o implementare eficientă, el aduce următoarea optimizare: atunci când trebuie să evalueze un număr de șiruri, el nu le redescoperă de fiecare dată, ci le folosește pe cele de la evaluările precedente.

Într-o manieră mai formală, șirurile descoperite de Mihai se pot determina folosind următorul model (numit în literatura de specialitate *gramatică independentă de context* - concret, aceasta reprezintă o mulțime de șabloane/patterns care permite generarea unor șiruri de caractere bazându-se pe anumite reguli):

```
A -> 1A | B
B -> B0 | ""
```

unde A și B sunt două simboluri externe (numite simboluri *neternale*), folosite pentru a putea fi înlocuite în continuare în mod recursiv, folosind setul de reguli de mai sus.

Spre exemplu, cea de-a doua regulă din gramatica de mai sus semnifică faptul că un simbol neterminal *B* (un simbol care poate fi înlocuit în continuare) poate fi înlocuit fie cu șirul vid, fie cu "B0", adică *B* poate genera, prin aplicări diferite ale regulilor de înlocuire, toate șirurile care conțin doar 0, cu oricâte zerouri.

Exemplu

Primele 10 șiruri obținute după regulile de mai sus sunt:

```
"" 1 0 11 10 00 111 110 100 000
```

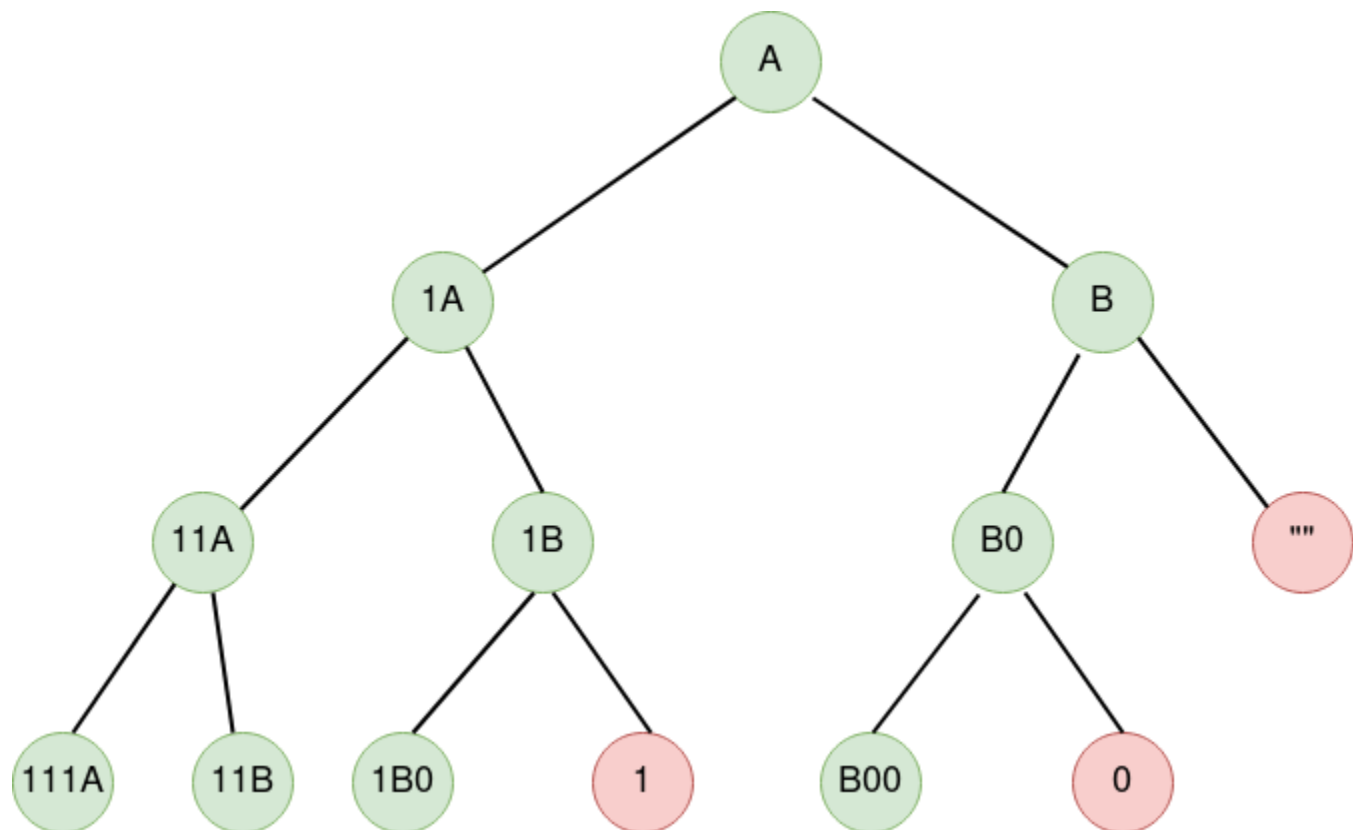
Să presupunem că programul nostru primește următoarele date de intrare:

```
2 3 7 4 -2
```

Atunci la ieșire se va afișa:

```
2 -> [ "", "1" ]
3 -> [ "", "1", "0" ]
7 -> [ "", "1", "0", "11", "10", "00", "111" ]
4 -> [ "", "1", "0", "11" ]
```

Aici este o imagine care evidențiază primele 4 nivele ale structurii de date modelată în problemă, numită arbore de derivare:



Din nefericire, codul scris de Mihai nu reușește să facă ceea ce el și-a propus. Ajutați-l pe Mihai să rezolve problemele și să facă primul pas înainte în proiectul său personal.

Download sursă [aici](#).